

TP5 — Complexité des Algorithmes et Recherche

Informations générales

- Filière : PASS Informatique — ESIEA
- Séance concernée : Séance 4 — Complexité des algorithmes, recherche
- Notions travaillées : notion de complexité, notation Grand O, recherche séquentielle, recherche dichotomique, tri à bulles
- Pré-requis : tableaux, boucles, méthodes (TP1 à TP4)
- Durée indicative : 1h30
- Remarque : ce TP n'utilise pas la récursivité, qui sera introduite en Séance 5

Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Expliquer ce qu'est la complexité d'un algorithme et pourquoi elle importe
- Reconnaître les classes de complexité $O(1)$, $O(n)$, $O(n^2)$ et $O(\log n)$ dans un extrait de code
- Implémenter une recherche séquentielle et une recherche dichotomique dans un tableau
- Comparer les deux méthodes de recherche selon leur complexité
- Implémenter un tri à bulles et évaluer sa complexité

Exercice 1 — Reconnaître la complexité d'un algorithme

Énoncé

Pour chacun des extraits de code suivants, indiquez sa complexité approximative en fonction de la taille n du tableau `tab` ($O(1)$, $O(n)$ ou $O(n^2)$), et justifiez brièvement.

Extrait A

```
System.out.println(tab[0]);
```

Extrait B

```
for (int i = 0; i < tab.length; i++) {  
    System.out.println(tab[i]);  
}
```

Extrait C

```
for (int i = 0; i < tab.length; i++) {  
    for (int j = 0; j < tab.length; j++) {
```



```
        System.out.println(tab[i] + tab[j]);
    }
}
```

Exercice 2 — Recherche séquentielle

Énoncé

Écrivez une méthode `rechercheSequentielle(int[] tab, int cible)` qui parcourt le tableau du début à la fin et renvoie l'indice de la première occurrence de cible, ou -1 si elle n'est pas trouvée. Testez-la sur le tableau {8, 3, 17, 5, 12, 9} en cherchant la valeur 12, puis la valeur 100.

Exercice 3 — Recherche dichotomique

Énoncé

La recherche dichotomique (ou recherche binaire) ne fonctionne que sur un tableau trié. Elle compare la cible à l'élément du milieu, puis élimine à chaque étape la moitié des éléments restants. Complétez la méthode ci-dessous.

```
static int rechercheDichotomique(int[] tab, int cible) {
    int debut = 0;
    int fin = tab.length - 1;

    while (debut <= fin) {
        int milieu = (debut + fin) / 2;

        if (tab[milieu] == cible) {
            return milieu;
        } else if (tab[milieu] < cible) {
            // ... a completer
        } else {
            // ... a completer
        }
    }
    return -1;
}
```

Exercice 4 — Séquentielle contre dichotomique

Énoncé

On cherche la valeur 72 dans le tableau trié {2, 5, 8, 12, 16, 23, 38, 45, 56, 72} (10 éléments).

- a) Combien de comparaisons la recherche séquentielle effectue-t-elle dans le pire des cas pour ce tableau ?
- b) Déroulez "à la main" la recherche dichotomique pour cette même valeur : indiquez debut, fin et milieu à chaque étape.

- c) Un tableau contient maintenant 1 000 000 d'éléments triés. Combien d'étapes, au maximum, la recherche dichotomique effectuera-t-elle environ (sachant que chaque étape divise la taille par 2, et que $2^{20} \approx 1\,000\,000$) ?

Exercice 5 — Le tri à bulles

Énoncé

Le tri à bulles compare deux éléments voisins et les échange s'ils sont dans le mauvais ordre, en répétant l'opération sur tout le tableau jusqu'à ce qu'il soit entièrement trié. Écrivez une méthode `triABulles(int[] tab)` qui trie un tableau d'entiers par ordre croissant, puis affichez le tableau {5, 1, 4, 2, 8} avant et après le tri.